

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR

INSTITUT UNIVERSITAIRE SAINT JEAN

SAINT JEAN INGENIEUR



INSTITUT SAINT JEAN

REPUBLIC OF CAMEROON

Peace-Work-Fatherland

MINISTRY OF HIGHER EDUCATION

SAINT JEAN UNIVERSITY INSTITUTE

SAINT JEAN ENGINEER



UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR

PROGRAMMATION ORIENTE OBJET (POO)

Rapport Technique :

SIMNet — Simulateur de Réseau Intelligent en Python

Travaux effectués du 12 Avril 2026 au 22 Mai 2026

**OPTION : Système Réseaux & Télécommunication Niveau 3
(SRT-3)**

Rédigé et présenté par :

BAKOP MKALE

NOUMBO MICHEL-ANGE

OUETE MARC

TAGUIWA BORIS

TCHATCHUM MIGUEL

Sous la supervision de :

M. FEDIM Stéphane

Année académique

2025/2026

1. Présentation du Groupe et des Rôles

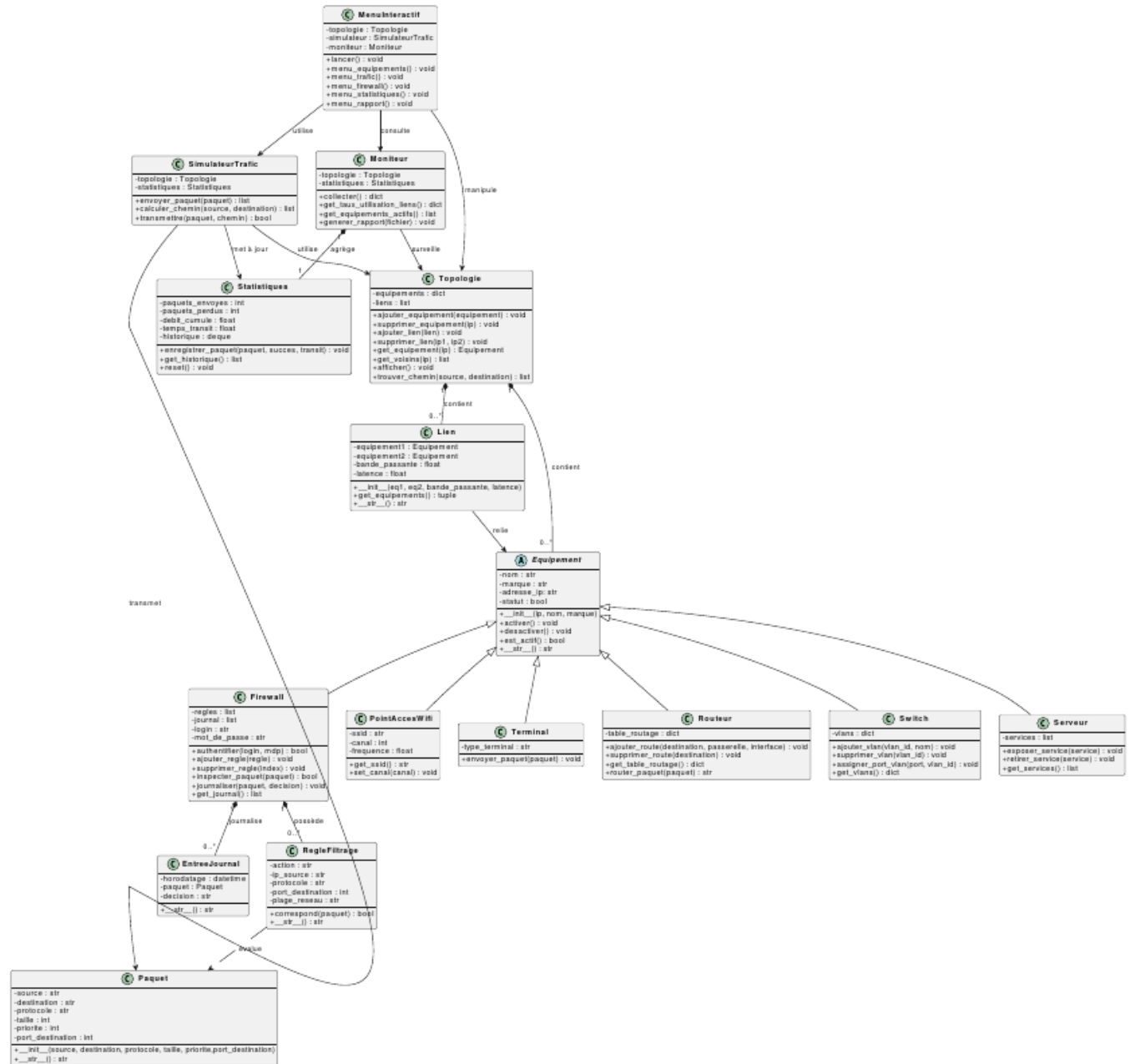
Notre groupe est composé de cinq étudiants en Ingénieur 3 SRT. Chaque membre a pris en charge un module fonctionnel du projet SIMNet, de la conception à l'implémentation.

Membre	Module assigné
OUETE Marc & NOUMBO Michel-Ange	Module 1 — Modélisation du réseau
BAKOP McKale & TAGUIWA Boris	Module 2 — Simulation de trafic
BAKOP McKale	Module 3 — Sécurité et filtrage
TAGUIWA Boris	Module 4 — Surveillance et rapports
TCHATCHUM Miguel	Module 5 — Interface console interactive

Chaque membre a contribué à son module via des commits Git réguliers sur la branche du groupe. La coordination s'est faite par échanges entre membres et revues de code avant intégration.

2. Diagramme de Classes Complet

Le diagramme suivant représente l'architecture complète du simulateur SIMNet. Il regroupe les 16 classes du projet, leurs attributs, leurs méthodes et les relations entre elles.



Description des classes

Classe	Fichier	Rôle
Equipement (ABC)	equipements.py	Classe abstraite mère de tous les équipements réseau
Routeur	equipements.py	Gère la table de routage et achemine les paquets
Switch	equipements.py	Gère les VLANs et l'assignation des ports
Serveur	equipements.py	Expose et retire des services réseau
Firewall	securite.py	Filtre les paquets, journalise les décisions, protégé par authentification
PointAccesWifi	equipements.py	Diffuse un réseau sans fil (SSID, canal, fréquence)
Terminal	equipements.py	Équipement utilisateur final, envoie des paquets
Lien	topologie.py	Relie deux équipements avec bande passante et latence
Topologie	topologie.py	Contient les équipements et liens, calcule les chemins (BFS)
Paquet	paquets.py	Représente un paquet circulant dans le réseau
Statistiques	paquets.py	Collecte les métriques de trafic, historique des 10 derniers paquets
SimulateurTrafic	paquets.py	Calcule les chemins et transmet les paquets saut par saut

RegleFiltrage	securite.py	Définit une règle AUTORISER/BLOQUER appliquée par le Firewall
EntreeJournal	securite.py	Entrée horodatée dans le journal du Firewall
Moniteur	moniteur.py	Surveille la topologie et génère le rapport d'exploitation
MenuInteractif	main.py	Interface console interactive — point d'entrée du simulateur

3. Justification des Choix de Conception

3.1 Abstraction

La classe Equipement est définie comme classe abstraite (ABC) avec les méthodes abstraites activer(), desactiver(), est_actif() et __str__(). Ce choix impose un contrat à toutes les sous-classes : elles doivent obligatoirement implémenter ces méthodes. Toute tentative d'instancier directement Equipement lève un TypeError, ce qui prévient les erreurs de conception.

- Les six sous-classes concrètes héritent toutes de ce contrat.
- Le module abc de la bibliothèque standard est utilisé, conformément aux consignes du sujet.

3.2 Héritage

Toutes les classes d'équipements héritent d'Equipement par héritage simple. Chaque sous-classe appelle super().__init__() pour initialiser les attributs communs, puis ajoute ses propres attributs spécifiques :

- Routeur ajoute table_routing (dict) pour gérer les routes.
- Switch ajoute vlans (dict) pour gérer les réseaux virtuels.
- Serveur ajoute services (list) pour les services exposés.
- Firewall ajoute regles, journal, login et mot_de_passe pour le filtrage sécurisé.
- PointAccesWifi ajoute ssid, canal et frequence pour la configuration Wi-Fi.
- Terminal ajoute type_terminal pour distinguer les postes utilisateurs.

3.3 Encapsulation

L'encapsulation est appliquée à plusieurs niveaux :

SIMNet — Simulateur de Réseau Intelligent en Python

- Les attributs d'Équipement sont protégés avec `_` (`_nom`, `_marque`, `_ip`, `_statut`).
- Les attributs `__login` et `__mot_de_passe` du Firewall sont privés (double tiret bas), accessibles uniquement via `authentifier()`.
- Dans `MenuInteractif`, les méthodes internes sont préfixées `_` pour distinguer l'interface publique des méthodes d'usage interne.

3.4 Polymorphisme

La méthode `__str__` est redéfinie dans chaque sous-classe d'Équipement. Un appel `print(equip)` produit un affichage adapté au type réel de l'objet. La fonction `isinstance()` est utilisée pour identifier dynamiquement chaque équipement lors du parcours de la topologie.

3.5 Surcharge d'opérateurs

Les méthodes spéciales `__str__` et `__init__` sont surchargées sur `Paquet`, `Lien`, `RegleFiltrage` et `EntreeJournal` pour un affichage naturel avec `print()` et une initialisation cohérente via `super().__init__()`.

3.6 Algorithme de routage — BFS

L'algorithme BFS (Breadth-First Search) est utilisé dans `Topologie.trouver_chemin()`. Il garantit le chemin le plus court en nombre de sauts, est simple à implémenter avec `collections.deque` et est parfaitement adapté à des topologies de taille d'entreprise.

4. Difficultés Rencontrées et Solutions Apportées

Difficulté rencontrée	Solution apportée
Imports circulaires entre les modules : <code>equipements.py</code> et <code>paquets.py</code> se référençaient mutuellement.	Réorganisation des classes dans chaque fichier en plaçant les classes sans dépendances en premier. Ordre d'import strict et documenté.
Gestion du cas où la source ou destination d'un paquet n'existe pas dans la topologie.	Vérification préalable dans <code>get_equipement()</code> . La méthode retourne <code>None</code> si introuvable. <code>SimulateurTrafic</code> enregistre alors le paquet comme perdu.

Coordination du travail à cinq personnes sur le même dépôt Git sans conflits.	Chaque membre a travaillé sur sa branche fonctionnelle. Les fusions vers la branche du groupe se font via Pull Requests après relecture.
Limiter l'historique des paquets aux 10 derniers sans gérer manuellement la suppression.	Utilisation de <code>collections.deque(maxlen=10)</code> : Python supprime automatiquement l'entrée la plus ancienne dès que la limite est atteinte.
Protéger l'accès à la configuration du Firewall sans système d'authentification externe.	Méthode <code>authentifier(login, mdp)</code> avec attributs privés <code>__login</code> et <code>__mot_de_passe</code> inaccessibles depuis l'extérieur de la classe.

Conclusion

SIMNet est un simulateur de réseau d'entreprise entièrement orienté objet, développé en Python 3 avec les seuls modules de la bibliothèque standard. Le projet met en œuvre les cinq concepts fondamentaux de la POO : abstraction, héritage, encapsulation, polymorphisme et surcharge d'opérateurs.

La conception fondée sur un diagramme de classes établi en amont a permis une répartition claire des responsabilités et une intégration fluide des modules. Le projet est livré fonctionnel, documenté et versionné conformément aux consignes de l'examineur. Nous avons rencontré des difficultés au niveau du Merge conflict, Problème de push.